Unsafe **M**ethod **O**f **C**ommunication



Adrian Schön, Lucas Mohr, Max Rohrhirsch, Pascal Lingenfelser

UMOC Team

Max Rohrhirsch (Backend / Blogs / Dev-Ops)

Pascal Lingenfelser (Backend, Blog-Kommentare)

Lucas Mohr (Frontend, Projektmanagement)

Adrian Schön (Frontend, Dokumentation)

Agenda

1. Projektziele
2. Architektur
3. Frontend-Implementation
4. Frontend-Design
5. Backend
6. Infrastruktur
7. Erkenntnisse
8. Live-Demo

1. Projektziele

Vision:

Moderner, webbasierten Messenger mit End-to-End-Verschlüsselung, „unsicheren“ Gamification Features und echter Chat-Erfahrung.

Projektziele

1. **Messenger-Grundlagen**: Grundlegende Chat-Funktionalitäten mit geringer Latenz

2. **Fun-Faktor**: Gamification durch unsichere Features

3. **Sicherheit**: Authentifizierung, Rollen- und Zugriffskontrolle

Begriffsdefinition

Unsafe vs. Unsecure

Gamification und Unsichere Features

Beispiele

- **Live-Preview**: Nachrichten sehen bevor sie Abgeschickt werden
- **Unsafe-Items**: Hintergrund ändern, Werbung anzeigen
- **Streak-Punkte und Items**: Punktesystem, Shop
- **Sonderfunktionen**: Blockierte Nutzer dürfen “Abschieds”-Nachricht senden

2. Architektur

Architekturentscheidungen

Komponente	**Möglichkeiten**	**Entscheidung und Begründung**
Echtzeit-Kommunikation	Websockets, HTTP-Long-Polling	**Websockets**: Bidirektionalität, geringere Latenz
Frontend-Framework	Angular, ReactJS, NextJS	**Angular**: Vorhandene Expertise
Backend-Framework	Flask, Django, Node.js, Spring	**Flask + SocketIO**: schnelle Prototypen, Python-Expertise)
Datenbank	SQLite, PostgreSQL	**SQLite:** Leichtgewichtig, Migration möglich

Qualitätsziele

1. **Benutzungsfreundlichkeit**: UI-Ladezeit < 2s, Intuitive Navigation

2. **Leistungseffizienz**: Nachrichtenbearbeitung in Echtzeit (Latenz < 500 ms)

3. **Sicherheit**: JWT-Auth, Transport-Verschlüsselt, Zugriffskontrolle

Tech-Stack

Frontend: Angular + NodeJS, Typescript

Backend: Flask (REST-API) + SocketIO (Events)

Datenbank: SQLite, ORM über SQLAlchemy

Auth & Security: JWT, TLS via Cloudflare

DevOps: Docker, Docker-Compose, Watchtower, GitHub Actions

Design Patterns

Layered Architecture: Presentation | Business Logic | Data Access

Observer Pattern: Event-Handling zwischen Client & Server

Repository Pattern: Saubere Trennung von Data-Access und Logik

3. Frontend-Implementation

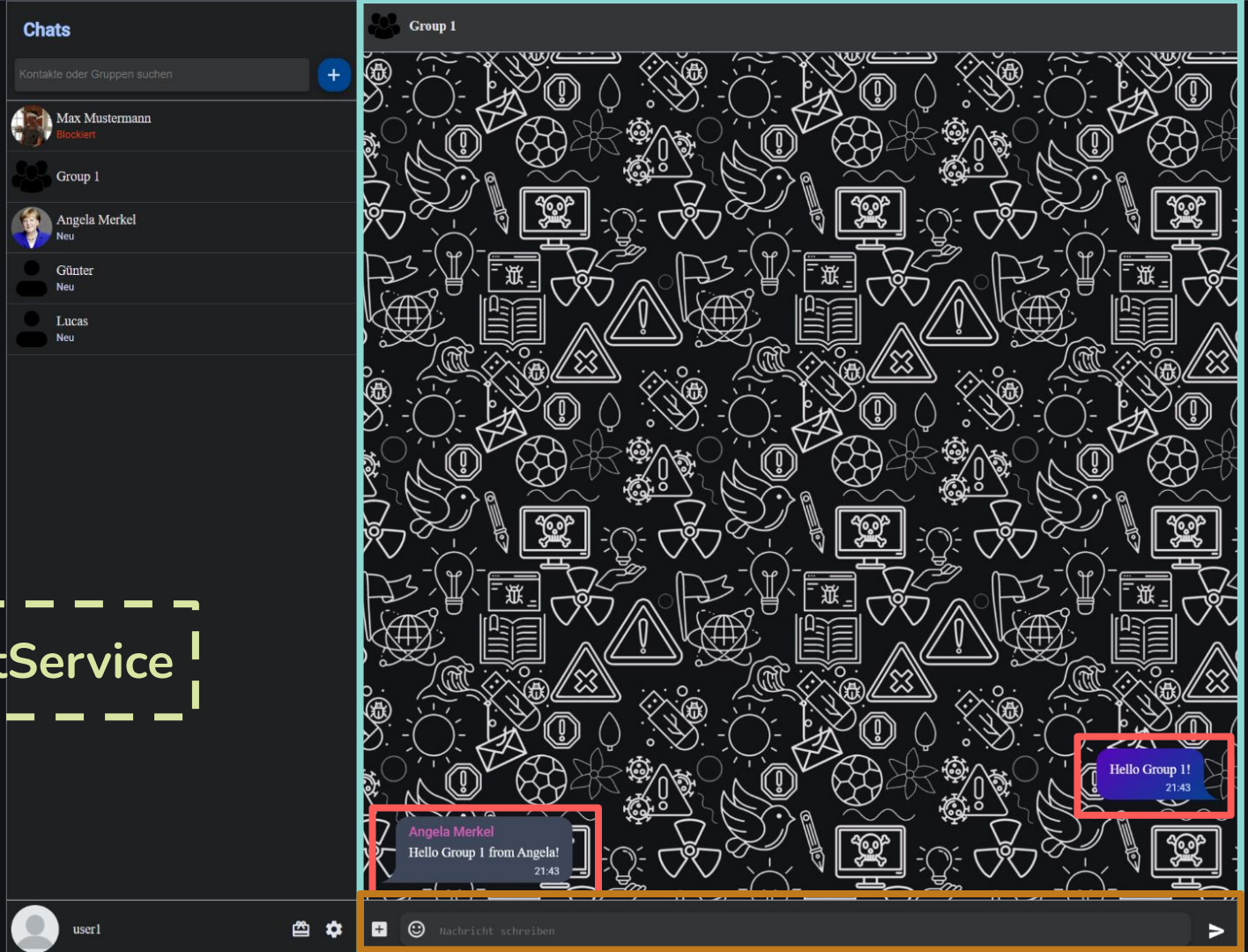
Komponenten-Architektur

Komponenten: Jeder Teil der App existiert als eigenständige Komponente

Komponenten-Hierarchie:

- **Container Components** (z.B. ChatWindowComponent) koordinieren Sub-Views und Services
- **Presentational Components** (z.B. MessageComponent) sind rein fürs Rendering zuständig
- **Feature Components** (z.B. ChatInputComponent) kapseln Spezial-UIs (Emoji-Picker)

Services: Logik und Netzwerkdienste



ChatService

State Management

Angular Signals: Werte welche alle Verbraucher über Updates informieren

Effekte: Callback bei Updates eines Signals

RxJS-Integration: HTTP Aufrufe in Services über Observables, dann in Signals umgewandelt

```
isLoading = signal(false);
self = signal<Contact | null>(null);
contacts = signal<(Contact | Group)[]>([]);
selectedContact = linkedSignal<(Contact | Group)[], (Co
  source: this.contacts,
  computation: (source, previous) => {
    if (previous?.value) {
      return (source.find((opt) => opt.contact_id === p
    }
    return null;
  }
});
effect(() => {
    if (!this.contactService.selectedContact()) return;
    this.chatService.isLoading.set(true);
    this.chatService.currentChatHistory.set([]);
    this.chatService.fetchChatHistory(
      this.contactService.selectedContact()!.contact_id
    );
    setTimeout(() => this.scrollToBottom(), 0);
  }
);
```

```
fetchChatHistory(contact_id: string) {  Show usages  Lucas Mohr
  this.http.get<any>(
    this.environmentService.getContactMessagesUrl(),
    {
      headers: new HttpHeaders({'Authorization': `Bearer ${this.loginService.getAuthToken()}`}),
      params: new HttpParams().append('chat_id', contact_id)
    }
  ).pipe(
    map(response => {
      const messagesData = Array.isArray(response) ? response : (response.messages ? response.messages : []);
      return messagesData.map((message: any) => {
        return new Message(
          message.content,
          message.message_id,
          message.recipient_id,
          message.sender_user_id,
          message.sender_username,
          new Date(message.timestamp),
          message.type
        );
      });
    })
  ).subscribe(
    newMessages => {
      this.currentChatHistory.set(newMessages);
      this.isLoading.set(false);
    }
  );
}
```

Echtzeit

WebSocket Events (Socket.IO v4)

- **Beispiel**: new_message, receive_char, item_used
- **Neue Nachricht**: Neue Nachrichten werden durch new_message angezeigt, ohne dass die Website aktualisiert werden muss
- **Live Preview**: Zeichen für Zeichen via receive_char-Event gestreamt
- **Item-Effekte**: Flashbang, Ads, Timeout werden per Event ausgelöst

4. Frontend-Design

Farb- und Typografie-Strategie

Angular Material 3, **Dark Mode** als Standard

Primärpalette: Azure für Buttons, Links, Active States

Tertiärpalette: Violet für Akzente, Tags

Systemfont: Roboto

Dynamic User Colors: Farben der User in Group-Chats per Hash-Funktion generiert

```
html {
  color-scheme: dark;
  @include mat.theme((
    color: (
      primary: mat.$azure-palette,
      tertiary: mat.$violet-palette
    ),
    typography: Roboto,
    density: 0
  ));
}
```

Responsive Design

Breakpoint-based approach mit begrenzter mobiler Optimierung

Hauptproblem: Size Warning Message

Werbung passt unter 750px nicht mehr auf die Website

```
<p class="size-warning" style="display: none;">
  BITTE VERGRÖßERE DAS BROWSER FENSTER,
  WIR HABEN NOCH KEIN GUTES RESPONSIVE DESIGN</p>
```

```
// Main responsive breakpoints
@media (max-width: 1000px) {
  .app-container {
    height: auto;
    min-height: 100vh;
  }

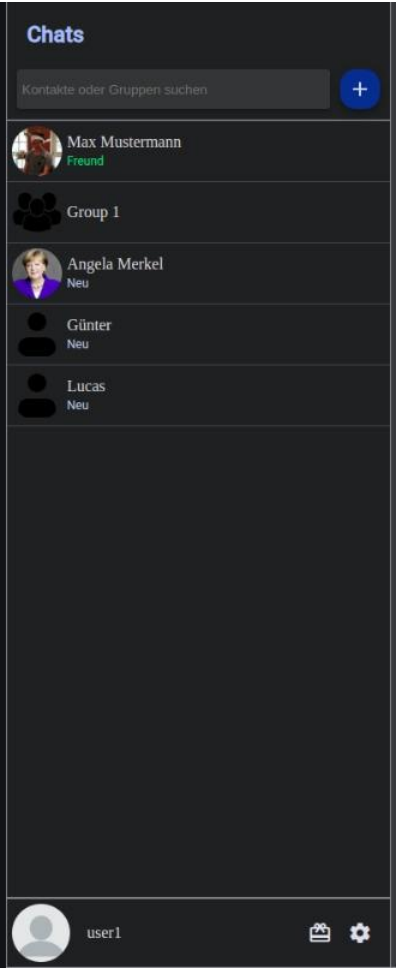
  .chat-area {
    order: 1;
    height: 50vw;
    max-height: 700px;
  }
}

@media (max-width: 750px) {
  .app-container {
    display: none !important;
  }
  .size-warning {
    display: inline !important;
  }
}
```

[IMAGE]

Hier könnte Ihre Werbung stehen!

[IMAGE]



UMOC

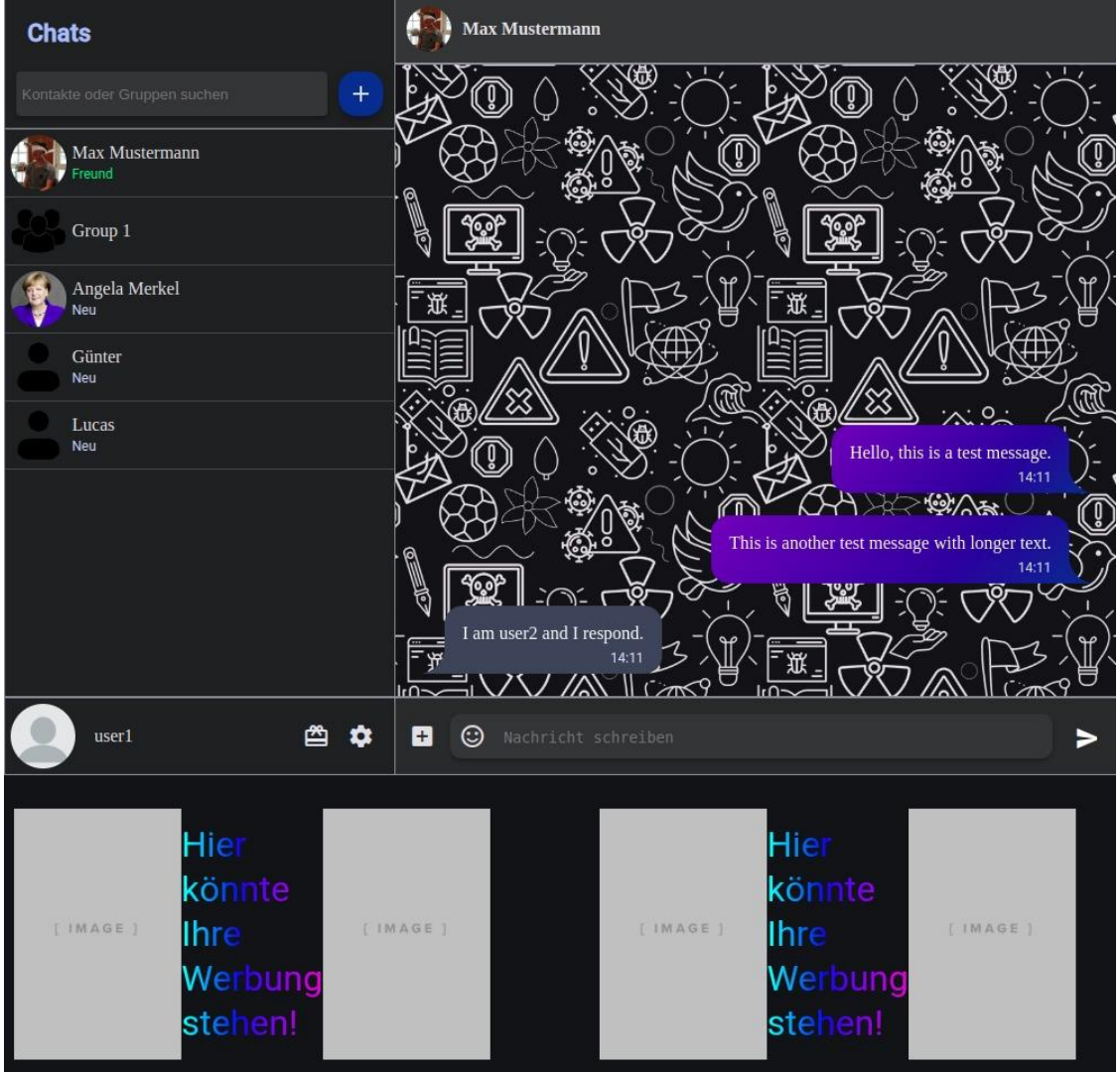
Unsafe Method of Communication

UMOC

[IMAGE]

Hier könnte Ihre Werbung stehen!

[IMAGE]


Chats
Kontakte oder Gruppen suchen
Max Mustermann
Freund
Group 1
Angela Merkel
Neu
Günter
Neu
Lucas
Neu
user1
Max Mustermann
Hello, this is a test message.
14:11
This is another test message with longer text.
14:11
I am user2 and I respond.
14:11
Nachricht schreiben
[IMAGE]
Hier könnte Ihre Werbung stehen!
[IMAGE]
[IMAGE]
Hier könnte Ihre Werbung stehen!
[IMAGE]


BITTE VERGRÖßERE DAS BROWSER FENSTER, WIR HABEN NOCH KEIN GUTES RESPONSIVE DESIGN

5. Backend

Ziel des Backend

Backend ist das Fundament von UMOC und erfüllt folgende Funktionen:

- Authentifizierung und Sicherheit
- Nachrichtenverarbeitung in Echtzeit
- Gruppen- und Kontaktverwaltung
- Integration des Item-Systems
- Skalierbarkeit und Wartbarkeit
- Haupt Logik der Anwendung wird durch das Backend bereitgestellt

Kombination von klassischen REST-Endpunkten mit WebSocket-Events für Flexibilität

Authentifizierung

JWT-basiert: Tokens für API-Zugriff und WebSocket-Authentifizierung

Session-Handling: Token Refresh und Logout-Logik

Sicherheitsmaßnahmen:

- Token-Verifizierung bei jedem Zugriff
- WebSocket-Zugriff nur mit gültigem Token

Messaging-System

Funktionen:

- **Direktnachrichten** (1 zu 1)
- **Gruppenchats** mit Adminfunktionen
- **Live Features**:
  - Echtzeit-Übertragung mit new_message event
  - Live-Preview mit receive_char event
- **Persistenz**:
  - Nachrichten werden dauerhaft gespeichert
  - Löschfunktion: Nachrichten können “gelöscht” werden

WebSocket Kommunikation

Client → Server

- **send_message**: Neue Nachrichten senden
- **send_char**: Einzelne Zeichen für Live-Preview
- **item_used**: Item aktivieren

Server → Client

- **new_message**: Neue Nachrichten empfangen
- **receive_char**: Tippanzeige der Live-Preview
- **item_used**: Item wurde aktiviert

Die Events werden über **JWT-authentifizierte Verbindungen** geschickt

Gruppen und Kontaktverwaltung

Gruppen:

- Erstellen, Verändern, Verlassen
- Admin-Rechte
- User hinzufügen / entfernen

Kontakte:

- Freundschaftsanfragen, Blockieren
- Statusmanagement: FRIEND, BLOCK, TIMEOUT, LAST_WORD

Teststrategie

Integrationstests: pytest-flask

API Response Time Testing: Locust für WebSocket-Lasttests

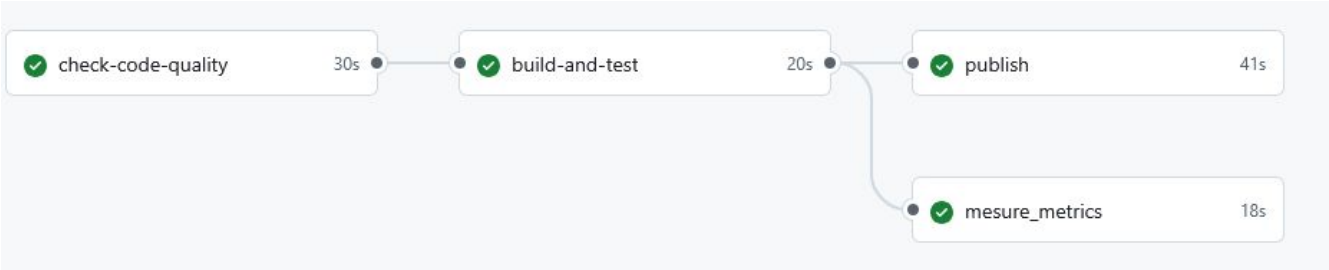
Codequalität:

- pylint
- Cyclomatic Complexity Analyse (radon, lizard)

6. Infrastruktur

CI/CD Pipeline

1. Codequalität prüfen via pylint; Score ≥ 80% als Mindestanforderung
2. Build & Test: Testdurchführung der Integrationstests
3. Codequalität analysieren:
   - Cyclomatic Complexity (radon), Response for Class (lizard)
   - Vorbereitungen für API Response Time Tests (locust, optional)
4. Publish (nur auf **main)**:
   - Docker Buildx Setup, Build des Images
   - Push zum GitHub Container Registry



Deployment-Architektur

Host:

- Raspberry Pi mit Docker-Compose

Tunnel und Security:

- Cloudflare Tunnel,Geo-Blocking

Monitoring:

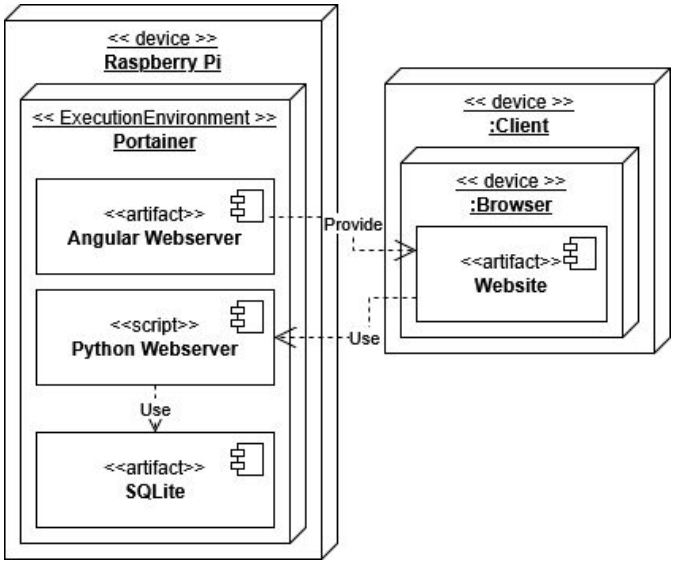
- Log-Aggregation (Docker Logs)

Deployment:

- Watchtower

Backup:

- Regelmäßige Image-Snapshots



7. Erkenntnisse

Lessons Learned

Scrum: 2-wöchentliche Sprints, Retros, Burndown Charts (Jira)

Kommunikation: Klare Rollen & regelmäßige Sync-Meetings notwendig

Timeboxing: Kritisch, um Feature Creep zu vermeiden

Refactoring: Regelmäßige Debt-Beseitigung erhöht Stabilität

Testing: Frühes Aufsetzen von Integrationstests zahlt sich aus

Ausblick und Weiterentwicklung

Neue unsichere Features:

- Als anderer User schreiben
- Subway Surfers Gameplay im Hintergrund

UX-Verbesserungen: Scaling, Accessibility, Responsive Design

Skalierung: Migration zu Cloud, Postgres

CI/CD: Blue/Green Deployment

Security: Continuous Pentesting, E2EE

Zusammenfassung: Vision erreicht

- Kombination aus Unsafe-Features & Security erfolgreich umgesetzt
- Stabile, wartbare Codebasis mit automatisierten Pipelines
- Nächste Schritte: neue Gamification-Module, UI Verbesserungen

8. Live-Demo

IP: 172.16.22.236:8000

Username: Vorname
Password: 123456789

Fragen?